

🔧 Introduction to Mechatronics — Revision Pack

Level 3 · Practical, Arduino-based course · theory + hands-on project code

Focus: this pack matches the practical nature of the course — the **mechatronics block diagram** plus complete, working **Arduino code** for each project you built: button press counter · attendance counter (LCD) · LCD text display · traffic light · LED count display · **seven-segment counter**.

Part A — Core Theory

1. What is Mechatronics?

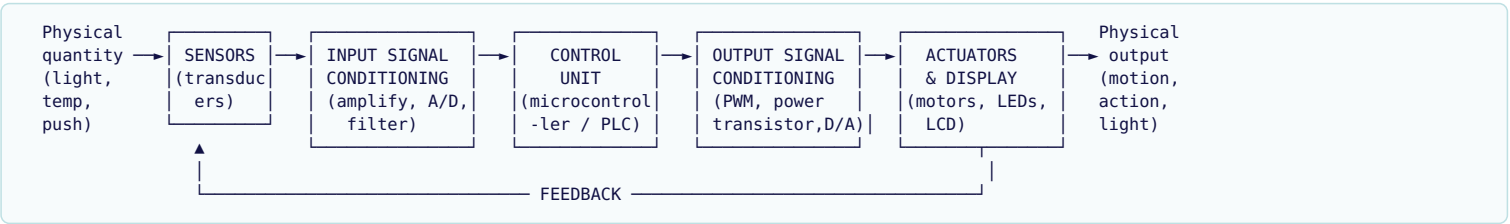
Mechatronics is the **synergistic combination of precision mechanical engineering, electronic control and systems thinking** in the design of products and manufacturing processes. It integrates: **mechanical · electronic · electrical · computer · control engineering**.

Advantages	Disadvantages
Cost-effective, good-quality products	High initial cost of the system
Achieves performance hard to get otherwise	Needs knowledge of several engineering fields
High flexibility; greater machine utilization	Each system's problems addressed separately
Intelligent self-correcting feedback → higher productivity & reliability	Expensive to retrofit into old systems

Applications: home appliances (washing machines), automobiles (electronic fuel injection, ABS), aircraft, CNC machines, robots, automated manufacturing.

2. The Mechatronics System Block Diagram

A mechatronic system follows a **measurement-and-control flow** from a physical input to a physical output, with **feedback**:



Block	Role	Examples
Sensors / transducers	Detect the state of system parameters & convert a physical quantity into an electrical signal	Switches, potentiometer, photoelectric, thermocouple, strain gauge, encoder
Input signal conditioning	Connects sensor to controller; cleans/converts the signal	Amplifiers, filters, A/D conversion
Control unit	The "brain" — processes inputs and decides outputs	Microcontroller (Arduino), SBC, PLC, logic circuits
Output signal conditioning	Connects controller to output devices; boosts the signal to drive them	Amplifiers, PWM , power transistors, D/A
Actuators	Produce motion or cause an action	DC/stepper/servo motors, solenoids, hydraulics, pneumatics
Display	Provide visual feedback to the user	LEDs, LCD , digital displays, CRT

Map to your Arduino projects: push-button = **sensor**; pull-down/debounce = **input conditioning**; Arduino = **control unit**; current-limiting resistor / transistor = **output conditioning**; LEDs & LCD = **display/actuators**.

3. Sensors & Transducers

A **transducer** detects the presence/magnitude/change of a physical quantity and gives a proportional electrical output. Classified by working principle: variable-resistance, variable-inductance, variable-capacitance, voltage-generating, and voltage-divider types.

- **Sensor** — detects/measures a physical input (light, heat, force, position, a button press).
- **Actuator** — converts the control signal into physical action (motor turning, LED lighting).

Part B — Arduino Fundamentals

Concept	Explanation
<code>void setup()</code>	Runs once at start — configure pins, start Serial/LCD
<code>void loop()</code>	Runs repeatedly forever — the main program
<code>pinMode(pin, MODE)</code>	Set a pin as <code>INPUT</code> , <code>INPUT_PULLUP</code> or <code>OUTPUT</code>
<code>digitalWrite(pin, HIGH/LOW)</code>	Turn an output pin on (5V) or off (0V)
<code>digitalRead(pin)</code>	Read a digital input → returns <code>HIGH</code> or <code>LOW</code>
<code>delay(ms)</code>	Pause for the given milliseconds
<code>Serial.begin(9600) / Serial.print()</code>	Open / write to the Serial Monitor

Edge detection & debounce (key for counters): a button "press" is detected when the state changes from LOW to HIGH (a **rising edge**) — comparing the current state to the previous state. A short `delay()` avoids counting one press many times (contact "bounce").

Part C — Project Code

Project 1 – Button Press Counter (Serial Monitor)

Button on pin 2, LED on pin 13. Counts each press and prints the total.

```
const int buttonPin = 2;
const int ledPin = 13;
int buttonState = 0;
int lastButtonState = 0;
int counter = 0;

void setup() {
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  buttonState = digitalRead(buttonPin);
  if (buttonState != lastButtonState) {      // state changed
    if (buttonState == HIGH) {               // button just pressed
      counter++;
      Serial.print("Pressed ");
      Serial.print(counter);
      Serial.println(" times");
    }
    delay(50);                               // debounce
  }
  lastButtonState = buttonState;
}
```

Project 2 — Student Attendance Counter (LCD)

16x2 LCD (rs,en,d4-d7 = 12,11,5,4,3,2), button on pin 7. Each press = one more student.

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

const int buttonPin = 7;
int count = 0;
int lastState = LOW;

void setup() {
  lcd.begin(16, 2);
  pinMode(buttonPin, INPUT);
  lcd.print("Attendance:");
}

void loop() {
  int state = digitalRead(buttonPin);
  if (state == HIGH && lastState == LOW) { // rising edge = new student
    count++;
    lcd.setCursor(0, 1); // second row
    lcd.print("Students: ");
    lcd.print(count);
    delay(200); // debounce
  }
  lastState = state;
}
```

Project 3 — LCD Display Words

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup() {
  lcd.begin(16, 2);           // 16 columns, 2 rows
  lcd.print("Hello, World!"); // row 1
  lcd.setCursor(0, 1);
  lcd.print("Mechatronics");  // row 2
}

void loop() { }              // nothing to repeat
```

Project 4 — Traffic Light

Red = pin 8, Yellow = pin 9, Green = pin 10. Classic Red → Green → Yellow cycle.

```
const int redPin = 8;
const int yellowPin = 9;
const int greenPin = 10;

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
}

void loop() {
  // --- RED (stop) ---
  digitalWrite(redPin, HIGH);
  digitalWrite(yellowPin, LOW);
  digitalWrite(greenPin, LOW);
  delay(5000);

  // --- GREEN (go) ---
  digitalWrite(redPin, LOW);
  digitalWrite(greenPin, HIGH);
  delay(5000);

  // --- YELLOW (get ready) ---
  digitalWrite(greenPin, LOW);
  digitalWrite(yellowPin, HIGH);
  delay(2000);
  digitalWrite(yellowPin, LOW);
}
```

Project 5 — LEDs Light Up on the Count

Button on pin 2; four LEDs on pins 8-11. Each press lights one more LED (a count bar); resets after all four.

```
const int buttonPin = 2;
const int ledPins[] = {8, 9, 10, 11};
const int numLeds = 4;
int count = 0;
int lastState = LOW;

void setup() {
  pinMode(buttonPin, INPUT);
  for (int i = 0; i < numLeds; i++)
    pinMode(ledPins[i], OUTPUT);
}

void loop() {
  int state = digitalRead(buttonPin);
  if (state == HIGH && lastState == LOW) { // new press
    count++;
    if (count > numLeds) count = 0; // reset after all lit
    for (int i = 0; i < numLeds; i++)
      digitalWrite(ledPins[i], (i < count) ? HIGH : LOW);
    delay(200); // debounce
  }
  lastState = state;
}
```

Project 6 — Seven-Segment Display Counter (likely exam task)

A 7-segment display is 7 LEDs (segments a-g). This shows 0-9 and increments each time a button is pressed. **Common-cathode** assumed: a segment lights when its pin is HIGH (for common-anode, invert with `!digits[...]`).

Segment layout:

aaa
f b
f b
ggg
e c
e c
ddd

a=2 b=3 c=4 d=5
e=6 f=7 g=8
button = 9
• dp (each segment via a 220 Ω resistor)

```
// Seven-segment 0-9 counter, incremented by a push button (common cathode)
const int segPins[7] = {2, 3, 4, 5, 6, 7, 8}; // segments a,b,c,d,e,f,g
const int buttonPin = 9;

// Which segments are ON for each digit {a,b,c,d,e,f,g} (1 = ON)
const byte digits[10][7] = {
  {1,1,1,1,1,1,0}, // 0
  {0,1,1,0,0,0,0}, // 1
  {1,1,0,1,1,0,1}, // 2
```

```

{1,1,1,1,0,0,1}, // 3
{0,1,1,0,0,1,1}, // 4
{1,0,1,1,0,1,1}, // 5
{1,0,1,1,1,1,1}, // 6
{1,1,1,0,0,0,0}, // 7
{1,1,1,1,1,1,1}, // 8
{1,1,1,1,0,1,1} // 9
};

int count = 0;
int lastState = LOW;

void displayDigit(int num) {
  for (int i = 0; i < 7; i++)
    digitalWrite(segPins[i], digits[num][i]); // light each segment
}

void setup() {
  for (int i = 0; i < 7; i++) pinMode(segPins[i], OUTPUT);
  pinMode(buttonPin, INPUT);
  displayDigit(0); // show 0 at start
}

void loop() {
  int state = digitalRead(buttonPin);
  if (state == HIGH && lastState == LOW) { // new press
    count++;
    if (count > 9) count = 0; // wrap 9 -> 0
    displayDigit(count);
    delay(200); // debounce
  }
  lastState = state;
}

```

Auto-count variant (0→9 every second): keep the same setup & `displayDigit()`, and use `void loop(){ for(int n=0;n<=9;n++){ displayDigit(n); delay(1000);} }`. The `digits[10][7]` lookup table is the core idea — each row = which segments form that number.

Common wiring reminders

- **Push button:** one side to 5V, other side to the input pin **and** through a **10 kΩ pull-down resistor** to GND (keeps the pin LOW when not pressed). Or use `INPUT_PULLUP` and wire the button to GND (logic inverts).
- **LED:** long leg (anode) → through a **220 Ω resistor** → digital pin; short leg (cathode) → GND. The resistor is the **output signal conditioning** that limits current.
- **LCD (16×2):** connect RS, E and data pins D4–D7 to the Arduino pins named in `LiquidCrystal lcd(...)`; add a potentiometer to V0 for contrast.